Traffic Light State Recognition — A2: SOTA Upgrade, Explainability and Web Demo

Gusmanov Albek · Computer Vision, Assignment 2 (continuation of A1) · Astana IT University · November 2026

1. Introduction and what changed since A1

A1 built a dataset of 421 traffic-light crops and trained three classifiers. A transfer-learned **ResNet18** was best (accuracy 0.922, macro-F1 0.902). A desktop app combined it with a **Faster R-CNN** detector. A1 also found two weaknesses. The model breaks down under blur and noise (macro-F1 ≈ 0.60), and its errors come mostly from **over-exposed lamps**: a red lamp with a white-yellow core gets called yellow. A2 keeps the same topic, data and leak-free split, and adds modern components that target exactly these problems:

Stage	A1 baseline	A2 (this work)
Detector	Faster R-CNN MobileNetV3 (2019)	YOLO11 (Ultralytics, 2024) — C3k2 + C2PSA attention
State classifier	ResNet18, A1 augmentation	ResNet18 + CondSim (final); YOLO11s-cls evaluated as an alternative
Robustness	mild Gaussian blur	CondSim: simulated night, glare, motion, noise+JPEG, low-res; rain and fog held out
Explanation	none	Grad-CAM heat-maps + SAM 2.1 promptable masks of the housing and the lit lamp
Application	tkinter desktop	Gradio web app: upload / camera → prediction → explanation, latency, environment

Every number in this report comes from the code in the repository. The experiments ran on a 2-thread CPU without a GPU. A1 and A2 use the same split, loss, optimiser, schedule and seeds, so the comparison is fair.

2. Dataset update

- **Real data (unchanged):** 421 crops: 190 green, 129 red, 102 yellow. Sources are S2TLD and the author's own photos. The exact A1 group-wise split (`split_a1.csv`) is reused: 258 train / 86 val / 77 test from 10 groups, with no group in two subsets. The class imbalance is handled with class-weighted cross-entropy, as in A1.
- **Synthetic share:** CondSim (Section 3.3) is applied on the fly with p = 0.5. On average **50 % of the training views in each epoch are synthetic variants** of real crops. Validation and the clean test set are **100 % real**. The robustness test sets are synthetic: 7 simulated conditions plus the 4 A1 perturbations, each applied to the 77 test crops (847 crops). They are labelled as synthetic everywhere.
- **New scene-level evaluation set:** 33 of the author's own full-resolution street photos (1920×2560, Astana), none of which appears in the crop log used for training. They contain **45 lit vehicle traffic lights** (35 yellow, 10 red) and **69 ignore regions** (unlit, back-facing, side-facing or pedestrian lights). Boxes were checked by eye from the union of the proposals of two detectors. A light that both detectors missed is therefore absent, and the set has **no green lights** (Section 7).

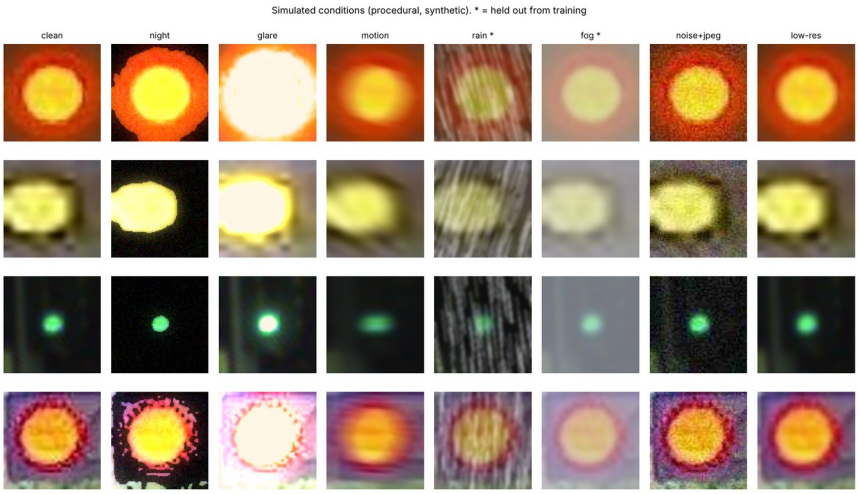


*Figure 1. CondSim on four test crops (strength 0.7). All images are procedural and synthetic. * = held out from training.*

3. SOTA methods

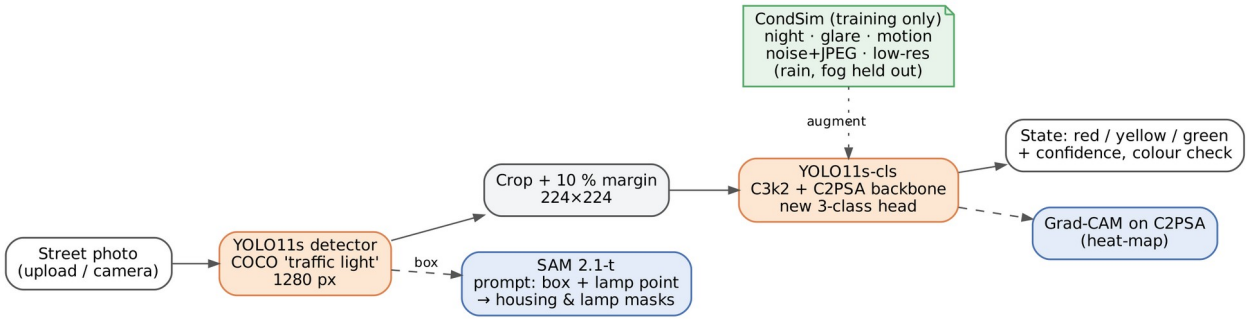


Figure 2. A2 pipeline. Orange = SOTA models; blue = explanation; green = training-time condition simulation.

3.1 YOLO11 — real-time detector (Ultralytics, 2024)

YOLO11 is the 2024 member of the one-stage YOLO family, whose lineage Neurohive covers from YOLOv4 [1]. Compared with YOLOv8, it replaces the C2f block with **C3k2** (a cheaper CSP bottleneck with two small convolutions). After SPPF it adds **C2PSA**, a block with position-sensitive spatial attention [2]. The authors say this attention helps with **small and partly occluded objects**, which matters here: a traffic light 100 m away is often only 10–30 px tall. YOLO11 predicts boxes in one pass without region proposals or anchors. We use the COCO-pretrained weights and the class *traffic light*, at 1280 px on a 1920×2560 photo, with no fine-tuning. On our photos it gives **fewer false boxes** than the A1 Faster R-CNN (Section 4.3). We also tried **YOLO11s-cls**, the classification version of the same backbone, as a state classifier.

3.2 SAM 2.1 — promptable segmentation (Meta, 2024)

SAM 2 [3] extends Segment Anything to video. It has a hierarchical image encoder, a prompt encoder (points, boxes, masks) and a mask decoder, plus a **memory attention** module and memory bank that carry the object from frame to frame. **MedSAM2**, recommended in the assignment, is a version of **SAM2.1-Tiny** fine-tuned for 3-D medical scans and video [4]. We use the same SAM 2.1-Tiny base without fine-tuning. It needs no traffic-light labels: two prompts are enough. The **detector box** gives the housing mask, and a **point on the brightest saturated pixel** gives the mask of the **lit lamp**. These masks are the visual explanation in the app. They are also the reference region for a quantitative Grad-CAM test (Section 5). The memory module would also let the masks follow a light through a dash-cam clip; this is future work.

3.3 CondSim — physically-motivated condition simulation

The assignment suggests text-to-video models (Step-Video-T2V) for drive-by clips. Step-Video-T2V has about 30 B parameters and needs a GPU with about 80 GB, which we do not have. Instead, we simulate the same physical effects procedurally: **night** (gamma darkening, the lamp stays bright, glow, sensor noise), **glare / blooming** (the lamp core clips to white with a coloured halo, i.e. the A1 failure mode), directional **motion blur**,

noise + JPEG and **low resolution**. **Rain** (streaks, contrast loss) and **fog** (airlight blending) are **never used in training** and serve as unseen conditions. Hue is never changed, because colour defines the class.

4. Experiments

Protocol. All models use the A1 recipe: class-weighted CE with label smoothing 0.05 and AdamW (weight decay 1e-4) with a cosine schedule. Training has two phases: the head only for 5 epochs at lr 1e-3, then full fine-tuning for up to 20 epochs at lr 1e-4. The best epoch and early stopping use validation macro-F1. Each configuration was trained with **2 seeds (42, 7)**; tables show mean ± std. The A1 weights were re-evaluated and reproduce A1 exactly (0.922 / 0.902).

4.1 Classifier: baseline vs SOTA (clean test, 77 crops)

Model	Params	Accuracy	Macro-F1	Yellow recall	Mean F1, 11 cond.	F1 rain+fog (unseen)
ResNet18 — A1 weights (baseline)	11.2 M	0.922	0.902	0.857	0.771	0.808
ResNet18 retrained, A1 recipe	11.2 M	0.922 ± 0.026	0.909 ± 0.031	0.857 ± 0.071	0.794 ± 0.004	0.810 ± 0.064
ResNet18 + CondSim (A2 final)	11.2 M	0.935 ± 0.013	0.922 ± 0.018	0.929 ± 0.071	0.827 ± 0.024	0.910 ± 0.004
YOLO11s-cls, A1 augmentation	5.4 M	0.916 ± 0.019	0.895 ± 0.027	0.929 ± 0.000	0.749 ± 0.025	0.824 ± 0.013
YOLO11s-cls + CondSim	5.4 M	0.864 ± 0.019	0.841 ± 0.020	0.857 ± 0.000	0.761 ± 0.003	0.820 ± 0.044

Table 1. Test metrics. "11 cond." = mean macro-F1 over 4 A1 perturbations + 7 simulated conditions.

The **final A2 classifier, ResNet18 + CondSim, is best or tied-best in every column**. Clean macro-F1 rises from 0.902 to 0.922 and yellow recall from 0.857 to 0.929. On rain and fog, which the model never saw, F1 rises from 0.81 to **0.91**. The SOTA backbone alone did **not** help: YOLO11s-cls is on par with ResNet18 on clean data (0.895 vs 0.909, within one std) at half the parameters and 1.8× lower CPU latency (32.9 vs 57.8 ms per crop). It was less robust, and CondSim *hurt* its clean accuracy. With 258 training crops, the larger ImageNet ResNet features seem to transfer more stably than the attention backbone. With seeds varying by ±0.02–0.03, differences below about 0.03 should not be over-read.

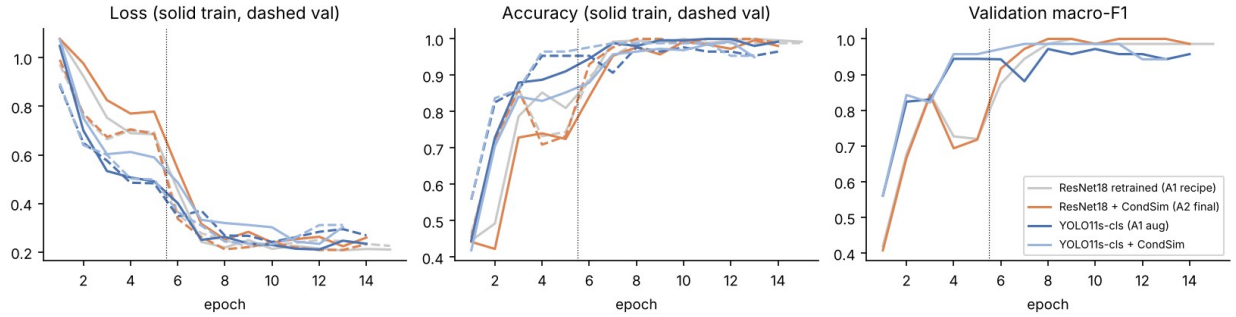


Figure 3. Training curves (seed 42). The dotted line marks the switch from head-only training to full fine-tuning.

4.2 Ablation: CondSim on / off

Metric (macro-F1)	ResNet18 w/o	ResNet18 with	Δ	YOLO11s-cls w/o	YOLO11s-cls with	Δ
Clean test	0.909	0.922	+0.013	0.895	0.841	−0.054
Mean over 11 conditions	0.794	0.827	+0.033	0.749	0.761	+0.012
Held-out rain + fog	0.810	0.910	+0.101	0.824	0.820	−0.004
A1 blur (σ = 3)	0.714	0.791	+0.076	0.500	0.652	+0.152
Simulated glare	0.689	0.783	+0.095	0.752	0.660	−0.091
A1 noise	0.599	0.624	+0.025	0.645	0.614	−0.031

Table 2. Ablation of CondSim (mean of 2 seeds).

For ResNet18, CondSim brings the largest gains on **glare** (+0.095), the A1 failure mode, and on the unseen rain/fog (+0.101), with no loss on clean data. Strong Gaussian **noise** stays the hardest condition for every model (≈ 0.6): our noise+JPEG simulation is milder than the A1 σ = 25 noise. For YOLO11s-cls the effect is mixed. The test set is small (14 yellow crops), so this negative result may partly be seed noise, but it shows that the augmentation must be tuned to the backbone.

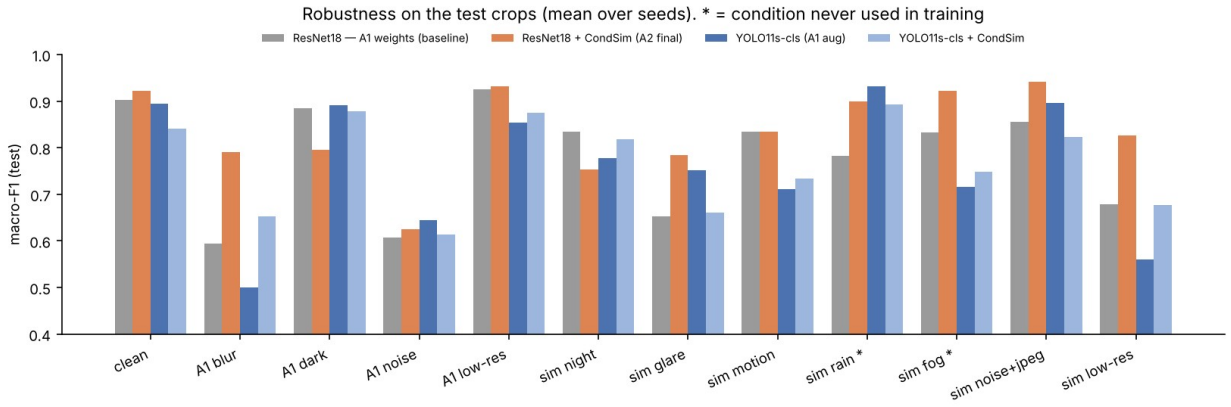


*Figure 4. Macro-F1 per condition (mean over seeds). * = condition never used in training.*

4.3 Detectors on full photos and end-to-end

Detector	AP@0.5	Recall @0.30	Precision @0.30	False boxes	CPU ms / image
Faster R-CNN (A1) @ 480	0.636	0.667	0.405	44	323
Faster R-CNN (A1) @ 960 — A1 app setting	0.813	0.822	0.529	33	1065
YOLO11n @ 1280	0.808	0.778	0.814	8	466
YOLO11s @ 1280 — A2 app setting	0.848	0.822	0.822	8	940
YOLO11m @ 1280	0.885	0.844	0.808	9	2649
YOLO11s @ 1920	0.874	0.867	0.696	17	2377

Table 3. Detection of lit traffic lights on 33 own photos (45 lights). Latency is the mean over 5 photos on an idle 2-thread CPU.

At the same CPU cost (≈ 1 s per photo), **YOLO11s raises AP@0.5 from 0.813 to 0.848 and precision from 0.53 to 0.82**: false boxes drop from 33 to 8, with the same recall. Higher resolution or the larger YOLO11m reaches AP 0.87–0.89 at 2.5× the cost. YOLO11n matches the A1 AP at less than half its time, so it is the choice for a phone. **End-to-end**, the full A2 pipeline (YOLO11s + ResNet18 + CondSim) labels **all 37 detected lights correctly** (100 %), and 82 % of the 45 lights are both found and labelled correctly. The full A1 pipeline (Faster R-CNN + ResNet18) also labels its 37 detections correctly, so on these daylight photos the end-to-end gain comes from the detector's 4× fewer false boxes; the classifier's gain shows on the test set and under hard conditions (Tables 1–2). Behind the YOLO11s detector, the A1 classifier mislabels one light (97.3 %) and the YOLO11s-cls alternative 9 of 37 (75.7 %), which confirms ResNet18 + CondSim as the final classifier. Recall is now limited by small, distant lights that no detector found at the 0.30 threshold.

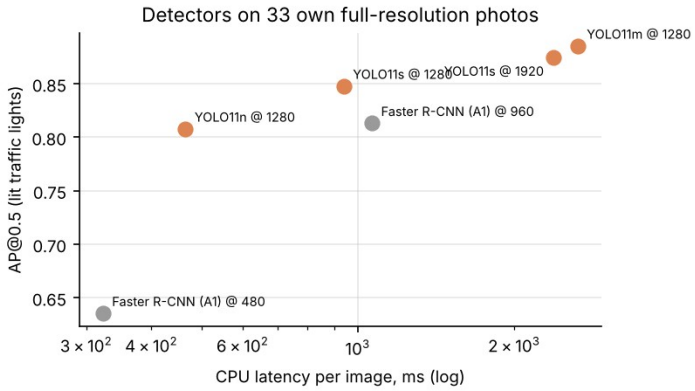


Figure 5. Accuracy–latency trade-off of the detectors (CPU).

5. Explainability

For every crop the app shows a **Grad-CAM** heat-map [5] (ResNet18: the last residual block; YOLO11-cls: the C2PSA block) for the predicted class, plus the **SAM 2.1** masks of the housing and the lit lamp. To go beyond pictures, we ran a **pointing game** [6] on the 68 test crops where SAM 2 produced a valid lamp mask. A hit means the Grad-CAM maximum falls inside the lamp mask dilated by 6 px; a random point would hit about 21 % of the time.

Model	Pointing-game hit rate	Grad-CAM mass inside lamp	Accuracy on these crops
ResNet18 — A1	0.63	0.34	0.912
ResNet18 + CondSim — A2	0.50	0.30	0.926
YOLO11s-cls	0.53	0.30	0.926

Table 4. Does the classifier look at the lit lamp? (SAM 2.1 lamp masks as reference; chance ≈ 0.21).

All models attend to the lamp far more often than chance. Surprisingly, the more accurate A2 model points *less often* into the lamp mask. Figure 6 shows why. On over-exposed red lamps, the A1 model looks at the **white-yellow core** and answers *yellow*. The CondSim model has seen many simulated blooms, so it looks at the **red halo around the lamp** and answers *red*. The halo lies outside the SAM lamp mask, so the pointing game counts a miss. Its evidence is still the physically correct one. So the pointing game is a useful sanity check but not a measure of *correct* reasoning, and the two explanations together show more than either does alone.

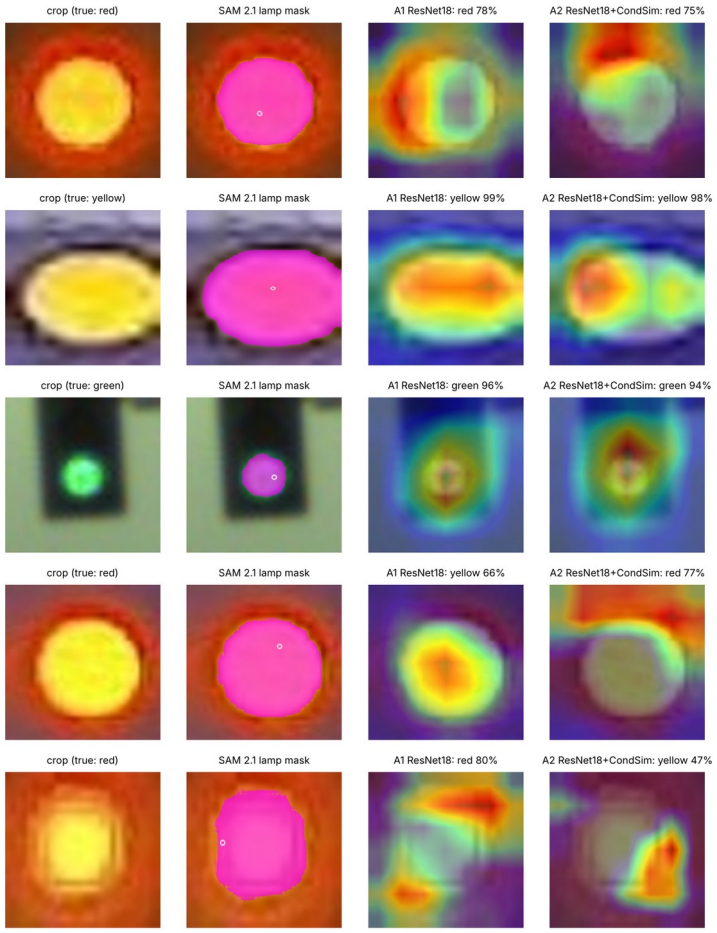


Figure 6. Crop → SAM 2.1 lamp mask (magenta; the circle marks the point prompt) → Grad-CAM of A1 and of A2. Row 4: A1 is wrong on an over-exposed red lamp and A2 is right (it looks at the halo). Row 5: a case where A2 is wrong.

6. Web application

The Gradio app (`app.py`, with an EN / RU language switch) has three tabs. **Recognise:** add a photo (upload, webcam / phone camera or one of the example photos) and press *Find traffic lights*. Each light gets a card with a small signal-head icon, the signal name, a confidence bar and a plain-language check. If the network and the pixel-colour check disagree, the card is dashed and marked “?”. Below the cards, *Why did the model decide this?* shows the Grad-CAM heat-map and the SAM 2 housing / lamp mask for each light. *Details* holds the class probabilities, the latency of each step, session averages and the environment (device, OS, PyTorch, browser). The detector, the classifier (A2, A1 or YOLO11s-cls, to compare live), the thresholds and the explanations are in a collapsed *Settings* panel. **Bad-weather simulator:** applies a CondSim condition to a close-up and compares all three classifiers on the same input, with a heat-map for each; the image is watermarked *SYNTHETIC*. **Results and ethics:** the key numbers, the disclosure from Section 7 and all result tables.

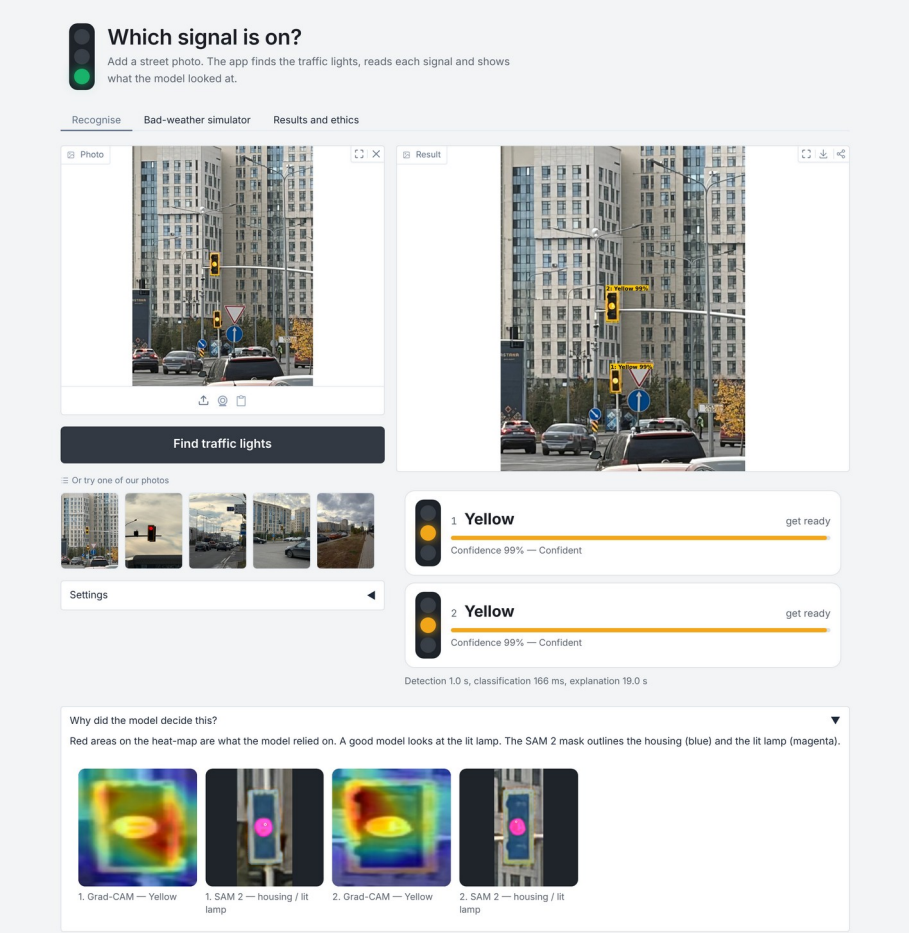


Figure 7. Recognise tab: YOLO11s + ResNet18 + CondSim on an own photo, with a result card, Grad-CAM heat-map and SAM 2 mask for each light (2-thread cloud CPU; the first SAM 2 call includes loading the model).

Measured latency (2-thread CPU, no GPU): detection ≈ 0.9–1.5 s per 5-MP photo, classification ≈ 60–170 ms per light, Grad-CAM ≈ 0.3–1 s and SAM 2.1-t ≈ 3–9 s per light. SAM 2 is therefore an optional checkbox. For a phone, YOLO11n plus a smaller classifier would be the deployment choice.

7. Ethics, disclosure and limitations

- **Not a safety system.** A wrong *green* is far more costly than a missed light. The app shows uncertain results as grey “?” and says plainly that it is a prototype that must not guide driving or crossing.
- **Synthetic media disclosure.** Every CondSim image is procedural (no generative model and no faces or people are synthesised). It is watermarked in the app and labelled as synthetic in this report, and its share of training (≈ 50 % of views) is stated in Section 2.
- **Consent and privacy.** The photos show public infrastructure taken from public places. Uploaded images are processed in memory and not stored. Cars, plates and pedestrians can appear in street photos: plates and faces should be blurred before results are published, and the app does not identify people.
- **Bias and coverage.** The data comes from a few cities, in daylight and dry weather, with Kazakh/Chinese signal designs. Yellow is the rarest class, and there is no snow, real night or rain. Real rain and night may differ from the simulation. The scene set has no green lights and only 45 lights. Its ground truth was built from detector proposals, which may favour the detectors; the 2-seed results have visible variance.
- **Misuse.** A state recogniser could feed automated enforcement or driving systems without validation. The code is released for education only.

8. Reproducibility

Environment: Python ≥ 3.10, PyTorch 2.x, ultralytics ≥ 8.3, gradio ≥ 5, grad-cam 1.5 (`requirements.txt`).
Weights: `python get_weights.py` downloads YOLO11n/s/m, YOLO11n/s-cls and SAM 2.1-t from the official

Ultralytics release. Faster R-CNN and ResNet18 ImageNet weights come from torchvision. The trained classifiers are in `models/`. **Inputs:** `dataset_grouped/` (A1), `results/split_a1.csv`, `images/full/` and `results/scene_gt.json`. **Commands:** `python app.py` (demo); `python -m tlcv.train_cls --model resnet18 --condsim --seed 42` (one run); `bash run_all.sh` (all runs); then `tlcv.det_eval`, `tlcv.det_latency`, `tlcv.e2e_eval`, `tlcv.xai_eval`, `tlcv.bench` and `tlcv.summarize`, which rebuild every table and figure in `results/`. The test perturbations are seeded per image, so results are deterministic.

9. Conclusion

A2 upgrades the A1 traffic-light system with a 2024 detector (YOLO11), promptable segmentation (SAM 2.1), Grad-CAM explanations and a web demo. The detector brings a clear win: +0.035 AP@0.5 and 4× fewer false boxes at the same CPU cost. For the classifier, physically-motivated condition simulation mattered more than a newer backbone. ResNet18 + CondSim improves clean macro-F1 (0.902 → 0.922), robustness to unseen rain and fog (0.81 → 0.91) and labels all detected lights in the field photos correctly. The explanation tools showed *how*: the A2 model reads the coloured halo of over-exposed lamps instead of their white core. The main next steps are real night and rain data, more yellow and green lights, and testing on video, using the SAM 2 memory to track lights between frames.

References

[1] Neurohive. “YOLO-v4 Is The New State-of-the-art Object Detector”. neurohive.io/en/news/yolo-v4-is-the-new-state-of-the-art-object-detector

[2] R. Khanam, M. Hussain. “YOLOv11: An Overview of the Key Architectural Enhancements”. arXiv:2410.17725, 2024; Ultralytics YOLO11, docs.ultralytics.com/models/yolo11

[3] N. Ravi et al. “SAM 2: Segment Anything in Images and Videos”. arXiv:2408.00714, 2024; github.com/facebookresearch/sam2

[4] Neurohive. “MedSAM2: Open Source SOTA 3D Medical Image and Video Segmentation Model” (built on SAM2.1-Tiny), 2025. neurohive.io/en/state-of-the-art/medsam2-…

[5] R. R. Selvaraju et al. “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization”. ICCV 2017; pytorch-grad-cam library.

[6] J. Zhang et al. “Top-down Neural Attention by Excitation Backprop” (pointing game). ECCV 2016.

[7] S2TLD: Small-Scale Traffic Light Dataset, Thinklab-SJTU. github.com/Thinklab-SJTU/S2TLD

[8] K. He et al. “Deep Residual Learning for Image Recognition”. CVPR 2016. S. Ren et al. “Faster R-CNN”. NeurIPS 2015.